

653. Two Sum IV Input is a BST

Problem Summary

Given the root of a BST and an integer k, return true if there exist two nodes whose values sum to k.

Algo

1. Initialize HashSet<Integer> set.
2. If root == null return false.
3. If set.contains(k - root.val) return true.
4. set.add(root.val).
5. Return findTarget(root.left, k) || findTarget(root.right, k).

Time Complexity & Space Complexity

O(n) — every node visited once.

O(n) — HashSet stores all node values in worst case.

Approach

DFS traversal with a HashSet. For each node, check if its complement (k - root.val) is already in the set. If yes → pair found. If no → add current value to set and continue.

Key Insights

- Same HashSet complement trick as **Two Sum (LC 1)** — k - root.val is the complement.
- Add to set AFTER checking — prevents using the same node twice (e.g., k=10, node value 5 → 5+5=10 should not count).
- Any traversal order works — pre/in/post/BFS all valid since you just need to visit every node.
- || short circuits — stops as soon as pair is found.

Approaches

- **Brute Force O(n²)**: For every pair of nodes check if they sum to k. O(n²) time.
- **Better O(n) space — Array**: In-order traversal → sorted array → two pointer approach. O(n) time O(n) space.
- **Optimal O(n) space — HashSet**: DFS with HashSet — for each node check if complement k - root.val already exists. O(n) time O(n) space.

Problem List

653. Two Sum IV - Input is a BST

Solved

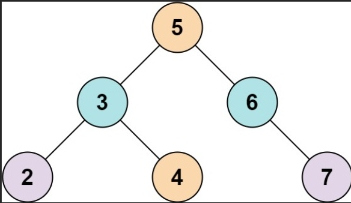
Easy

Topics

Companies

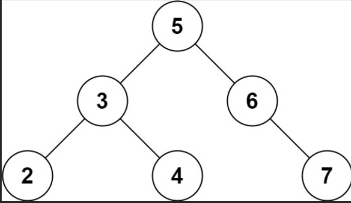
Given the root of a binary search tree and an integer k, return true if there exist two elements in the BST such that their sum is equal to k, or false otherwise.

Example 1:



Input: root = [5,3,6,2,4,null,7], k = 9
Output: true

Example 2:



Input: root = [5,3,6,2,4,null,7], k = 28
Output: false

7.4K

137

32 Online

Accepted

423 / 423 testcases passed

Sahil Khan submitted at Aug 03, 2026 11:18

₹583.25/mo Save over 55%

Unlock the Full LeetCode Experience

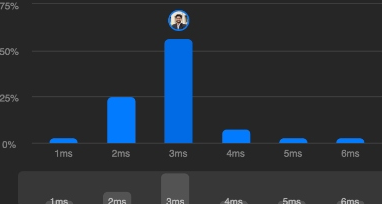
Company problems, Ask Leet, and expert editorials

Runtime

3 ms | Beats 73.06%

Memory

47.54 MB | Beats 10.15%



Code | Java

```
1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int val) { this.val = val; }
9  * }
```

View more

Code

Auto

```
8 *   TreeNode(int val) { this.val = val; }
9 *   TreeNode(int val, TreeNode left, TreeNode right) {
10 *       this.val = val;
11 *       this.left = left;
12 *       this.right = right;
13 *   }
14 * }
15 */
16 class Solution {
17
18     HashSet<Integer> set = new HashSet<>();
19
20     public boolean findTarget(TreeNode root, int k) {
21         if(root == null) return false;
22
23         if(set.contains(k - root.val)) return true;
24
25         set.add(root.val);
26
27         return findTarget(root.left, k) || findTarget(root.right, k);
28     }
29 }
30
```

Saved

Ln 16, Col 17

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

root =

[5,3,6,2,4,null,7]

k =